Gitlab air-gapped Install

This setup is tested on ubuntu 22.04 jammy and gitlab-ce_17.1.0

Requirements

- Clone server of same version of ubuntu but with internet access to download resources
- gitlab-ce
- Docker is needed for the runner

Downloading the setup

These steps should be run on the Clone server that has internet access:

```
curl --silent
"https://packages.gitlab.com/install/repositories/gitlab/gitlab-
ce/script.deb.sh" | sudo bash
sudo apt-get clean
sudo apt-get install --download-only gitlab-ce
sudo cp /var/cache/apt/archives/*.deb /path/to/save/setup
```

These commands should download gitlab-ce package to **/path/to/save/setup** along with any needed dependancies for the specific Ubuntu version

Installing Gitlab

1. Files should be copied from **/path/to/save/setup** to the actual offline server
2. Update /etc/hosts

```
echo "35.185.44.232 docs.gitlab.com
172.65.251.78 gitlab.com
104.18.4.247 embed.diagrams.net" | sudo tee /etc/hosts
```

This adds DNS entries that the setup requires to know even if it does connect to them 3. Install Gitlab

```
sudo EXTERNAL_URL="http://my-host.internal" dpkg -i gitlab-ce*.deb
```

Replace my-host.internal with the real URL it will respond to (or IP). Note this has to be http during the setup or gitlab will try to generate a certificate from LetsEncrypt servers

4. Get the default root password:

```
sudo cat /etc/gitlab/initial_root_password
```

5. Open the URL in the browser to login with username **root** and the password from the file (step 4)
6. Valid NGINX example:

```
  location / {
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header Host $http_host;
      proxy_set_header X-Forwarded-Proto $scheme;
      proxy_redirect off;
      proxy_http_version 1.1;
      proxy_connect_timeout      10m;
      proxy_send_timeout         10m;
      proxy_read_timeout         10m;
      proxy_pass http://IP-OF-GITLAB;
      client_max_body_size 3000M; ### this is just in case
}
```

Setting Up the runner

1. Docker needs to be installed on the runner machine (can be gitlab machine)
2. Get runner docker image on a server with internet connection and docker installed:

```
docker pull gitlab/gitlab-runner:latest
docker save gitlab/gitlab-runner:latest -o gitlab-runner.tar
```

3. After copying the gitlab-runner.tar to the runner server:

```
docker load -i  gitlab-runner.tar
```

4. Prepare the runner folder structure:

```
sudo mkdir -p /opt/hosting/runner_dir
sudo chown $USER:$USER /opt/hosting -R
cd /opt/hosting/runner_dir
```

5. Configure the runner:
    - Open Gitlab UI
    - Click **Admin Area** bottom left
    - On the Left Menu: **CICD** > **Runners**
    - **New Instance Runner**
    - Set Tag and **Create**
    - Take a copy of the run command specially *--url* and *--token*

6. Back to the server run the following:

```
docker run -it --rm  \
  -v $PWD/config:/etc/gitlab-runner \
  -v /var/run/docker.sock:/var/run/docker.sock \
  gitlab/gitlab-runner:latest register --url ***** --token ****
```

Follow the setup on screen (executor type should be docker and image can be anything available) Edit the config.toml

```
sudo vi config/config.toml
```

Add the following under runners.docker

```
    allowed_pull_policies=["if-not-present","always","never"]
    extra_hosts = ["host:ip"] #### if used the --add-host in the run
```

Start up the runner with the configuration:

```
docker run -d --name gitlab-runner --restart always \
  -v $PWD/config:/etc/gitlab-runner \
  -v /var/run/docker.sock:/var/run/docker.sock \
  gitlab/gitlab-runner:latest
```

Note: --add-hosts "Host:IP" can be added to the commands if Gitlab URL's DNS record is not known.

Build images:

Any needed image must be pulled same as the runner image to used on the server.

```
### on internet connected server
docker pull needed_image:tag
docker save needed_image:tag -o needed_image_tag.tar
### on offline runner server
docker load -i needed_image_tag.tar
```

Or it can be downloaded then uploaded to any private local registry to be pulled (this is more reliable when managing multiple runners on different servers)

```
### on internet connected server
docker pull needed_image:tag
```

```
docker tag needed_image:tag registryURL/needed_image:tag
docker push needed_image:tag ### this might require login based on the
setup
```

Typical images are: (depending on the needs)

- ubuntu:latest
- docker:latest
- docker:dind
- node:20
- mcr.microsoft.com/dotnet/sdk:6.0

Finish

This concludes the setup as well as all the required images for successfull builds